Making Industry Compilable: The Digital Reasoning Thread

Vlad Larichev*
Association Industrial AI (AI²)
Independent research, Germany
Vlad.Larichev@ai2n.eu

Dr. Dirk Molitor
Association Industrial AI (AI²)
Independent research, Germany
Dirk.Molitor@ai2n.eu

Michael Taylor
Siemens AG
Germany
taylormichael@siemens.com

* Corresponding author

Version v1 · November 2026 · CC BY 4.0
Extends Larichev & Molitor, *Digital Reasoning Thread* whitepaper v0.2 (2025)

ABSTRACT

The digital thread has long connected industrial data across the product lifecycle, yet the reasoning behind decisions remains fragmented across tools, teams, and time. Transformer-era generative systems—language models, multimodal and vision–language–action models, world-model-style predictors, and rich simulators—now propose industrial hypotheses and actions at scale, while industrial practice still depends on deterministic validators such as CAD checks, simulation, finance rules, policy, and safety gates. We argue that a Digital Reasoning Thread (DRT), a persistent and auditable layer of reasoning artifacts spanning humans, agents, and systems, can bind those proposers to validators so that industrial processes become compilable, traceable, and governed in the sense familiar from software builds: a fail-closed check and transform path with explicit evidence obligations, composing existing tools. Building on our earlier formulation of DRT, this paper states the compilability thesis, sketches architecture and requirements for that binding layer, and develops a synthetic engineering-change example in which missing evidence blocks or escalates action. The work is a step toward an industrial compiler rather than a report of a completed implementation.

https://doi.org/10.5281/zenodo.24056066

Keywords: Industrial AI; Digital Thread; Digital Reasoning Thread; Compilability; Governance; Traceability; Agentic systems; Manufacturing

1. INTRODUCTION

Industrial digitalization spent decades building the *digital thread*: a connective tissue for requirements, models, manufacturing data, and service records across the lifecycle [1], [2]. That achievement is real. What it does not make first-class is the reasoning that turns data into decisions—goals, assumptions, rejected alternatives, evidence obligations, and the path from hypothesis to approved action.

Software engineering faces a related problem and answers it with compilers and continuous integration. Source changes are checked, transformed, and either accepted or rejected with an explicit log. Industry already has local analogues—CAD rule checks, simulation campaigns, financial controls, retrieval from enterprise systems—and human processes such as V-model reviews. What is missing is an end-to-end compile of cross-domain decisions: a governed path that binds proposals to validators and leaves a traceable record when evidence is incomplete.

A second shift sharpens the gap. Transformer-era generative systems [3] now propose actions cheaply: language models drafting plans, multimodal and vision–language–action systems for physical settings,

world-model-style predictors, and high-fidelity simulators. Those systems are stochastic proposers. Industrial acceptance still rests on deterministic validators. Without an explicit binding layer, organizations either block automation or accept opaque chains of tool calls.

We argue that the Digital Reasoning Thread (DRT) [4] can serve as that layer. DRT treats reasoning artifacts as industrial assets—persistent, context-aware, and auditable across humans, agents, and systems. In this paper we state a compilability thesis: industrial processes become *compilable* when proposals travel with explicit context and evidence obligations through a fail-closed check and transform path, composing existing tools rather than replacing them. We outline architecture and requirements for that path and illustrate it with a synthetic engineering-change scenario. The long-range target is an industrial compiler; the present contribution is the position and the requirements sketch.

2. FROM DIGITAL THREAD TO DIGITAL REASONING THREAD

2.1. What the digital thread provides

Reviews of the digital thread emphasize end-to-end linkage of product and process artifacts, configuration control, and lifecycle analytics [1], [2]. Digital twins complement that picture by providing executable representations of assets and processes [5]. Interoperability roadmaps for distributed manufacturing stress standards and scalable integration rather than monolithic platforms [6]. In new product development, scalable AI adoption likewise depends on data quality, interoperability, and context management across domains [7].

These lines of work strengthen the flow of *what* is known. They do not by themselves make the *why* of a decision portable when ownership crosses PLM, MES, ERP, simulation, and human review.

2.2. Digital Reasoning Thread

We adopt the definition from our prior whitepaper [4]: DRT is a persistent, context-aware, and auditable representation of reasoning artifacts—goals, constraints, assumptions, evidence, plans, and traces—spanning humans, agents, and systems. The whitepaper distinguishes content generation, tool orchestration, and reasoning; insists on context engineering so assumptions survive handovers; and separates interaction, knowledge, reasoning, and control concerns in a thin reasoning plane.

A complementary public framing organises industrial AI capabilities into seven layers from sensing to governance. In that view the scarce asset is not another point tool but a vertical reasoning thread across layers—especially where reasoning and inference remain thin. Figure 1 summarises that stack and the role of DRT; it is consistent with, not a replacement for, the whitepaper's interaction / knowledge / reasoning / control split.

DRT does not replace the digital thread or the twin. It adds the rationale substrate those systems lack when agents and humans must justify, replay, or refuse an action.

3. WHAT "COMPILABLE" MEANS

In software, compilation lowers a program into a checked artifact or returns errors. The useful properties are not magic correctness of the world, but explicit contracts, static and dynamic checks, fail-closed behavior under missing prerequisites, and a build log that can be audited.

We use *compilable* for industrial decisions in that sense. A decision context is compilable when:

1. the intent, assumptions, and evidence obligations are explicit enough to check;
2. validators (CAD, simulation, finance, policy, safety, retrieval) can be invoked as part of a defined path;
3. missing or contradictory evidence yields block or escalate rather than silent improvisation;

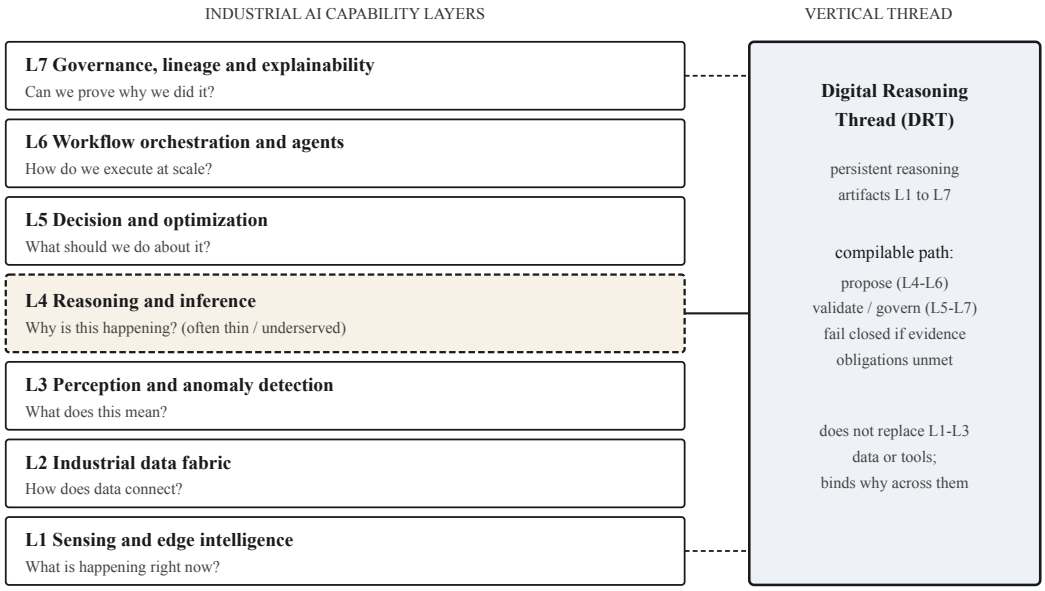


Figure 1. Seven-layer industrial AI stack with DRT as vertical connective tissue. Layer labels follow the public landscape narrative; L4 (reasoning) is often underserved. Compilability binds propose/validate/govern steps across the upper layers while referencing lower-layer data.

4. the resulting plan, or the refusal, is recorded as a traceable decision path.

Local compile-like tools already exist. The thesis is end-to-end binding: proposals from generative systems and humans enter the same fail-closed path. The metaphor has limits. Compilation does not prove that a plant state is safe in the physical world; it proves that declared obligations were discharged or that the action was refused.

4. RELATED WORK

Digital thread and twin. Lifecycle connectivity and twin characterisations are mature research topics [1], [5], [6]. Our claim is complementary: connectivity of artifacts is necessary but not sufficient for compilable decisions.

Agentic orchestration. Industrial GenAI deployments increasingly use orchestrators and specialized agents [8]. Orchestration moves work between tools. It does not automatically persist reasoning artifacts

Software	Industrial (via DRT)
Source ⟶ compile ⟶ artifact or errors	Intent + context ⟶ check/transform ⟶ plan or block/escalate
Types and static checks	Evidence obligations, contracts, policy gates
CI and build governance	Traceable, governed decision path
Compiler, linker, tests	CAD, simulation, finance, retrieval, MES, twin/sim environments
Generative coder + tests	Stochastic proposer + deterministic validators, bound by DRT

Table 1. Software compilation compared with industrial compilability via DRT.

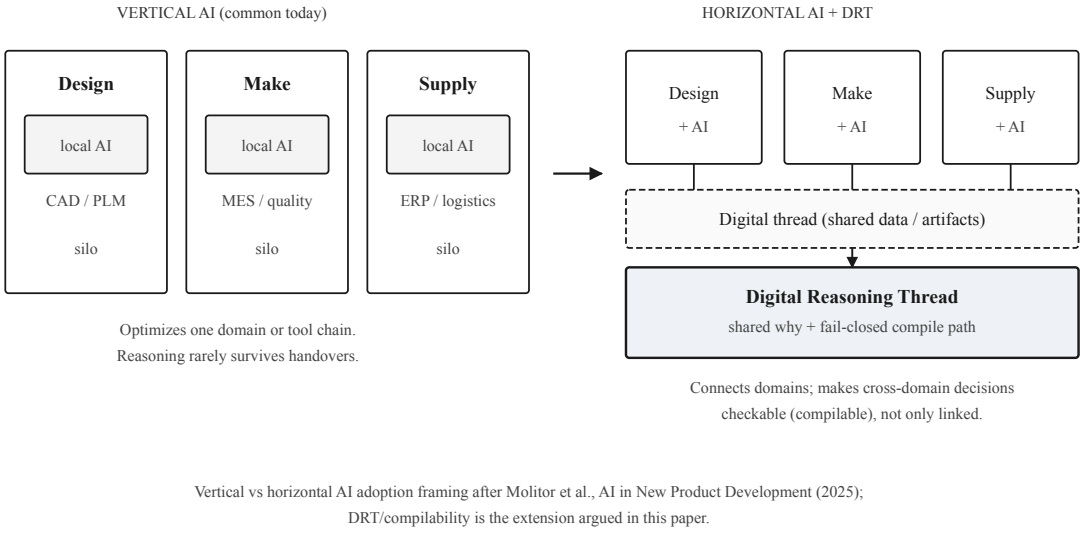


Figure 2. Vertical AI optimises within a domain silo. Horizontal AI connects domains via shared data (digital thread) and, in our extension, shared reasoning under a fail-closed DRT compile path.

or enforce fail-closed evidence obligations. We treat orchestration as a proposer/executor mechanism that still needs a reasoning and control plane.

Horizontal AI in engineering. Frameworks for AI in new product development distinguish vertical use cases (optimising a single process or domain) from horizontal ones that connect data, tools, and disciplines [7]. Most organisations still concentrate on the vertical pattern. Figure 2 contrasts that pattern with a horizontal setup in which the digital thread shares artifacts and DRT carries checkable reasoning across domains. Compilability operationalises part of the horizontal need as fail-closed evidence obligations on the path from proposal to action.

Generative and physical AI stacks. Language models, multimodal action models, and simulator-centric environments enlarge the set of proposers. We do not privilege a single model class. The architectural requirement is the binding to deterministic validators under an auditable thread.

5. ARCHITECTURE AND REQUIREMENTS

We retain the whitepaper separation of interaction, knowledge, reasoning, and control [4] and state requirements for compilability. Figure 3 sketches the binding role of DRT between stochastic proposers and deterministic validators atop the digital thread.

R1 — Explicit decision context. Each compile attempt names intent, scope, assumptions, and the set of required evidence objects (documents, model versions, sensor windows, approvals).

R2 — Proposer interface. Humans and generative systems may submit candidate plans or hypotheses. Submissions are recorded as reasoning nodes with inputs, outputs, and confidence or uncertainty markers where available.

R3 — Validator interface. Domain tools are invoked as checks with machine-readable pass, fail, or inconclusive results. Inconclusive and fail map to block or escalate according to policy.

R4 — Fail-closed control. The control plane encodes risk tiers, approval gates, and escalation targets. Missing obligated evidence is not treated as success.

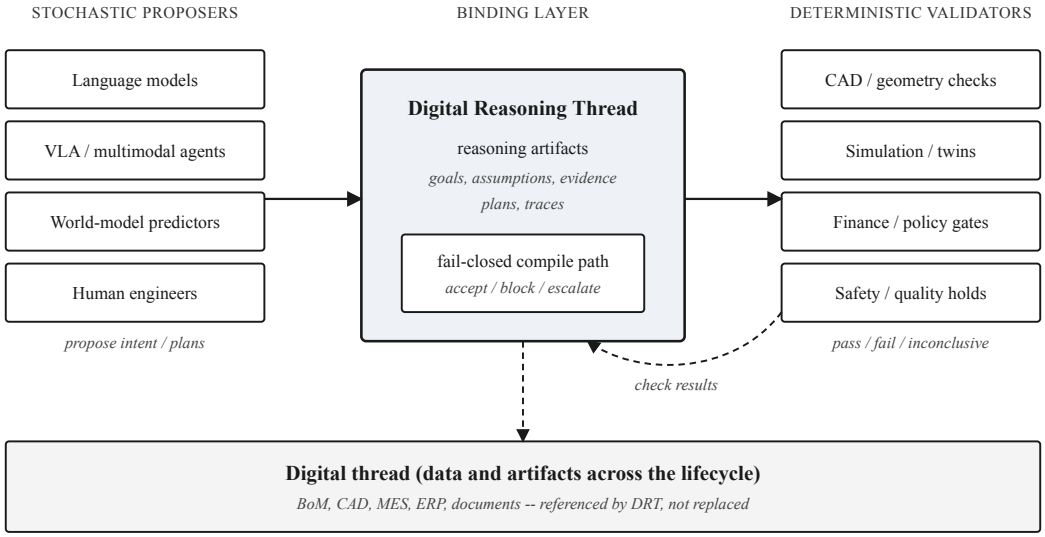


Figure 3. Stochastic proposers and deterministic validators bound by a Digital Reasoning Thread. The digital thread remains the data and artifact backbone; DRT carries reasoning artifacts along a fail-closed compile path.

R5 — Traceable decision path. Every compile attempt yields a thread segment that can be replayed: who or what proposed, which checks ran, which evidence was bound, and whether the outcome was accept, block, or escalate.

R6 — Composition, not replacement. DRT references existing digital-thread objects and enterprise tools. It does not require a new system of record for product data.

R7 — Tool neutrality. Bindings may use existing industrial buses and APIs; the requirements are semantic, not vendor-specific.

These requirements are proposed. They are intended to guide implementations and evaluation substrates, including synthetic environments such as DRT Loom [9], without treating any current repository as a completed industrial compiler.

6. SYNTHETIC WORKED EXAMPLE: ENGINEERING CHANGE WITH MISSING EVIDENCE

We use a synthetic engineering-change request (ECR) on a mechanical assembly. The scenario is a specification of desired behavior, not an executed experiment.

Setting. An engineer (or an agent acting on a prompt) proposes substituting a fastener class on a safety-relevant joint to improve assembly time. The digital thread holds the BoM, CAD, and prior change history. Validators available in the environment include CAD constraint checks, a simplified structural simulation, a cost service, and a quality-hold policy service.

Happy path. The proposer submits intent and assumed load case. DRT records the decision context and obligated evidence: current CAD revision, simulation report for the joint, supplier approval status, and a quality sign-off for safety-relevant changes. Validators return pass. The control plane accepts a provisional plan: update BoM, regenerate work instructions, notify MES. The thread segment stores rationale, rejected heavier fastener options, and validator outputs.

Missing-evidence path. The same proposal arrives without a simulation report for the revised joint and without supplier approval for the alternate fastener. Under R3–R5, compilation fails closed: the

action plan is not released to MES. The thread records the incomplete obligations and escalates to a human approver with the precise gaps. An agent is not permitted to "complete" the change by inventing missing evidence.

The example is deliberately narrow. It shows the difference between orchestration (calling tools) and compilability (refusing to proceed when obligated checks are absent).

7. TOWARDS AN INDUSTRIAL COMPILER

An industrial compiler, in our usage, is a system that repeatedly runs the check and transform loop over bounded industrial decision contexts, composing validators already present in the enterprise. DRT supplies the reasoning and control requirements for that loop. Synthetic evaluation environments can stress missing-evidence and multi-hop cases without exposing customer data [9]. Empirical studies, schema stabilization, and domain packs (manufacturing release, supply-chain exception, maintenance) remain future work.

8. LIMITATIONS

This paper presents a conceptual architecture, requirements, and a synthetic worked example. We do not report runtime measurements, controlled user studies, or a deployed compiler. The programme framing is independent research under the Association Industrial AI; author affiliations are listed on the title page and do not imply institutional endorsement of the claims. Safety in high-stakes settings continues to depend on policy coverage and human approval design, as noted in the earlier whitepaper [4].

9. CONCLUSION

The digital thread connected industrial data. Generative systems now propose actions at scale, while deterministic validators remain the gate to industrial acceptance. A Digital Reasoning Thread can bind proposers to validators so that processes become compilable, traceable, and governed in the software sense of fail-closed checks and auditable paths. We stated that thesis, listed requirements, and illustrated a missing-evidence escalation in a synthetic engineering change. The path ahead is to realize those requirements in an industrial compiler and to evaluate them on synthetic and, later, operational grounds.

REFERENCES

[1] Q. Zhang, J. Liu, and X. Chen, "A Literature Review of the Digital Thread: Definition, Key Technologies, and Applications," *Systems*, vol. 12, no. 3, p. 70, 2024, doi: 10.3390/systems12030070.

[2] Y. Zhang, W. Dong, J. Wang, C. Che, and L. Li, "The Development of Digital Thread: The Relations to Digital Twin and Its Industrial Applications," *Digital Transformation and Society*, vol. 1, no. 2, pp. 147–160, 2022, doi: 10.1108/dts-06-2022-0023.

[3] A. Vaswani *et al.*, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[4] V. Larichev and D. Molitor, "Digital Reasoning Thread: A Unified Reasoning Layer for Industrial Applications." [Online]. Available: https://github.com/vlarichev/digital-reasoning-thread/blob/main/pdf/digital_reasoning_thread.pdf

[5] D. Jones, C. Snider, A. Nassehi, J. Yon, and B. Hicks, "Characterising the Digital Twin: A Systematic Literature Review," *CIRP Journal of Manufacturing Science and Technology*, vol. 29, pp. 36–52, 2020.

[6] T. Hedberg, M. Helu, and T. Sprock, "A Standards and Technology Roadmap for Scalable Distributed Manufacturing Systems," in *ASME 2018 International Manufacturing Science and Engineering Conference*, 2018. doi: 10.1115/MSEC2018-6550.

[7] D. A. Molitor, V. Larichev, T. Guggenberger, M. Altendeitering, D. Porta, and M. Ziegler, "AI in New Product Development: Connecting Data & Unlocking Knowledge," technical report, 2025.

[8] V. Larichev, J. Masek, P. Chouhan, and D. Spiess, "Generative AI and Agentic Architecture in Engineering and Manufacturing: Potentials and Practice of Scalable AI Solutions," *ZWF Zeitschrift für wirtschaftlichen Fabrikbetrieb*, vol. 120, pp. 17–23, 2025, doi: 10.1515/zwf-2024-0166.

[9] Association Industrial AI, "DRT Loom." [Online]. Available: https://github.com/association-industrial-ai/drt-loom